

Optimizing kernel Parallelization for Shallow Neural Network Training: A Comparative Study

Authored by:

CHATTAH SALSABILA
BADAOU IGRAM

MARMOUZE NOR ELHOUDA
BERKAT CHERAZ

École nationale Supérieure d'Informatique (ESI) – 2025/2026

Abstract

This paper presents a performance analysis and optimization study of a CUDA-based shallow neural network training pipeline. Starting from a naive baseline relying on synchronous global memory operations, we propose an alternative strategy integrating persistent device memory, asynchronous CUDA streams, and tiled shared memory kernels. Experimental evaluation on an NVIDIA T4 GPU demonstrates up to $1.74\times$ speedup over the baseline, reducing large-dataset training time from 31.05s to 17.82s. Results indicate PCIe latency as the main bottleneck. Finally, we discuss potential further optimizations within the backpropagation phase

1 Introduction

The rapid growth of deep learning has made GPU-accelerated training a critical research area [1]. Shallow neural networks (SNNs), despite their simplicity, remain widely used in regression tasks due to their computational regularity and suitability for controlled performance studies.

Training involves repeated matrix multiplications during forward and backward propagation, making these operations the dominant computational bottleneck. While CUDA can theoretically exploit massive GPU parallelism, naive implementations often fail to fully utilize hardware resources due to inefficient memory access patterns, synchronization overhead, and suboptimal data transfer strategies.

This work investigates the performance gap between a naive CUDA baseline and a structured alternative strategy. Our contributions are: a systematic bottleneck analysis of the baseline, a device-centric alternative combining three optimizations, and an experimental comparison across multiple dataset scales on an NVIDIA T4 GPU.

2 Overview of the Reference Approach

2.1 Problem Definition and Learning Objective

The reference approach considers a supervised regression problem based on a shallow neural network. Given an input feature vector $X \in \mathbb{R}^n$, the model aims to predict a continuous output $\hat{Y}$ approximating the target value Y .

The learning objective is to minimize the prediction error over the training dataset by optimizing the network parameters Θ (weights and biases). This objective is formally expressed as:

$$\min_{\Theta} \mathcal{L}(Y, \hat{Y}),$$

where $\mathcal{L}$ denotes the loss function measuring the discrepancy between predicted and true outputs [2].

2.2 Neural Network Model Description

2.2.1 Model Type and Overall Architecture

The reference approach relies on a shallow feed-forward neural network (SNN) composed of an input layer, a single hidden layer, and an output layer.

The input layer receives the feature vector X , the hidden layer learns intermediate representations, and the output layer produces the predicted value $\hat{Y}$.

The network follows a strictly forward computation flow, without recurrence or feedback connections. The use of a single hidden layer allows the model to capture non-linear relationships while preserving a simple and regular structure. As a result, the computational workload is mainly dominated by dense linear algebra operations, making this architecture particularly suitable for controlled performance analysis and parallel execution studies [2].

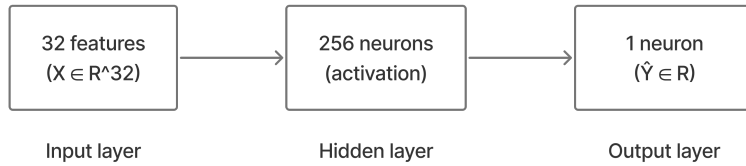


Figure 1: Architecture of the Shallow Neural Network (SNN)

2.2.2 Mathematical Formulation

Let $X \in \mathbb{R}^{m \times n}$ denote a batch of input samples, where m is the batch size and n the number of input features. The forward propagation through the network is defined as follows:

$$\begin{aligned} Z_1 &= X \cdot W_1 \\ H &= \text{ReLU}(Z_1) \\ \hat{Y} &= H \cdot W_2 \end{aligned}$$

where W_1 and W_2 are the weight matrices of the hidden and output layers respectively, $\text{ReLU}(x) = \max(0, x)$ is the activation function applied element-wise, and H represents the activations of the hidden layer.

The prediction error is evaluated using the Mean Squared Error (MSE) loss function:

$$\mathcal{L}(Y, \hat{Y}) = \frac{1}{m} \sum_{i=1}^m (Y_i - \hat{Y}_i)^2$$

The model’s computation is dominated by matrix multiplications and element-wise operations, which informs optimization strategies [2].

2.3 Dataset Generation and Training Configuration

The dataset is provided in CSV format, where each row represents one training sample. The first 32 columns correspond to input features, and the last column contains the target value. Data is loaded into memory and organized in matrix form to enable mini-batch training.

To study the effect of problem size on computational behavior, three dataset scales are considered while keeping the model and training parameters unchanged. The dataset specifications are summarized in Table 1. Training is performed using mini-batch stochastic gradient descent (SGD) with a fixed batch size of 256, a constant learning rate, and a predefined number of epochs.

Dataset	Samples	Batches
Small	256	1
Medium	2560	10
Large	25600	100

Table 1: Dataset Specifications [2]

2.4 Training Workflow

The network is trained using mini-batch Stochastic Gradient Descent (SGD). The dataset is partitioned into fixed-size batches, and the training procedure follows an iterative process. For each mini-batch, the following steps are executed:

1. Extract a mini-batch $(X_{\text{batch}}, Y_{\text{batch}})$ from the dataset.
2. Perform forward propagation to compute the predicted outputs $\hat{Y}$.
3. Compute the loss using the Mean Squared Error (MSE) over the current batch.
4. Perform backward propagation to compute the gradients of the loss with respect to the network parameters.
5. Update the model parameters using SGD by applying the computed gradients.

This sequence is repeated for all batches and across multiple epochs, resulting in a regular and repetitive computation pattern suitable for performance evaluation and optimization,[2].

3 Parallelism Characteristics of the Approach

3.1 Parallel Programming Models Used

The reference implementation exploits parallelism using two different programming paradigms depending on the target architecture. On multicore CPUs, parallelism is achieved using the POSIX threads (Pthreads) library, which enables explicit thread-level parallelism. On GPUs, the implementation relies on NVIDIA CUDA, which follows a massively parallel execution model based on threads, thread blocks, and grids [3].

This dual implementation allows the same algorithmic structure to be executed on distinct hardware architectures, enabling a comparative study of parallel execution strategies.

3.2 Data-Level Parallelism in Neural Network Computations

The parallelization strategy primarily relies on data-level parallelism, which naturally arises from the structure of neural network computations. Operations such as neuron activations, weighted sums, and gradient computations are applied independently across neurons or training samples.

As a result, the workload can be decomposed into multiple independent tasks that can be executed concurrently, making the approach well suited for both multithreaded CPU execution and GPU acceleration.

4 Parallelization Strategies

4.1 Multicore CPU (Pthreads)

The Pthreads implementation exploits coarse-grained data parallelism by partitioning the output matrix along the row dimension [4]. Each thread is assigned a contiguous subset of rows to compute independently, effectively eliminating write conflicts. Synchronization barriers are placed at the end of each parallel region to ensure data consistency before the next training step proceeds.

4.2 GPU Acceleration (CUDA)

The CUDA implementation leverages fine-grained SIMT parallelism [5], where the workload is distributed across thousands of threads.

- **Thread Mapping:** Each thread computes a single output element $C[i, j]$ by performing the dot product of row i of matrix A and column j of matrix B .
- **Execution Model:** Threads are organized into a 2D grid of blocks covering the output matrix dimensions. The host (CPU) manages memory allocation and triggers synchronous data transfers, while the device (GPU) executes the compute-bound kernel using global memory accesses.

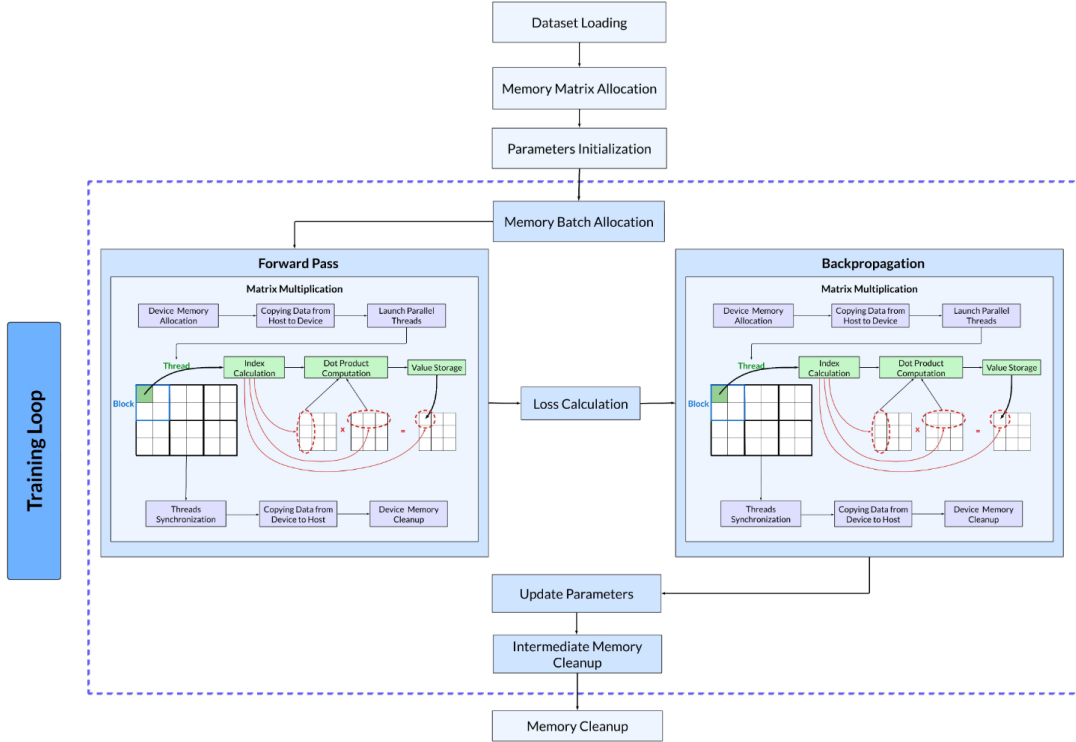


Figure 2: CUDA strategy: One thread per output element [2]

5 Critical Analysis and Limitations of the Baseline

The baseline implementation provides a functional framework for training a neural network using CUDA as a mathematical co-processor. However, our analysis reveal several structural design assumptions that introduce significant performance bottlenecks.

5.1 Memory Access Inefficiency and Bandwidth Saturation

The memory architecture of the baseline model is the primary inhibitor of performance:

- **Global Memory Latency:** Each thread calculates a single element of the output matrix by fetching operands directly from global memory. For each iteration of the inner product, the thread performs two global memory reads to retrieve elements from matrices A and B. This results in $2 \times A_cols + 1$ total global memory accesses per thread.
- **Non-Coalesced Access Patterns:** Threads within a warp do not always access contiguous memory addresses (specifically for the second operand). This prevents the hardware from grouping transactions into a single coalesced memory cycle.

5.2 Synchronization and Communication Overhead

The control logic in the baseline implementation suffers from frequent execution stalls:

- **PCIe Bottleneck:** Device memory allocation and data transfers occur inside the training loop for every batch. This introduces massive latency on the PCIe bus .
- **Blocking Synchronizations:** The use of `cudaDeviceSynchronize()` and synchronous memory copies forces the CPU to wait for the GPU to complete every task. This prevents any overlap between computation and data transfer.
- **Host/Device Barriers:** Operations such as ReLU activation and MSE loss calculation are performed on the CPU. This necessitates constant Device-to-Host transfers, leaving the GPU idle during host-side processing.

6 Positioning and Objectives of Our Work

Based on the limitations identified in the critical analysis, our work focuses on a "Device-Centric" approach. The objective is to transition from a naive co-processor model to an optimized one where the GPU handles the majority of the workload by implementing :

1. **Reduce Memory Bottlenecks:** Minimize global memory access latency and PCIe transfer overhead to improve throughput.
2. **Enhance GPU Utilization:** Leverage parallelism, streams, and shared memory to keep the GPU fully occupied, even for large-scale datasets.
3. **Optimize Computational Kernels:** Increase arithmetic intensity via tiling and kernel fusion to minimize global memory traffic. Utilizing pre-transposed data layouts ensures coalesced memory accesses, aligning with the GPU's row-major hardware architecture to maximize effective bandwidth.
4. **Minimize CPU-GPU Coordination Overhead:** Reduce blocking synchronizations and host-side computations to maintain a smooth computational pipeline.

7 Overview of the Proposed Optimizations

The following architectural enhancements were implemented to maximize throughput and minimize latency.

7.1 Cache Locality

- **Pre-transposition:** The weight matrix is transposed on the host prior to device transfer. This ensures that threads within a warp access elements from both the input and weight matrices in a contiguous, row-major fashion. This alignment facilitates

memory coalescing, allowing the hardware to combine multiple memory requests into a single transaction, thereby maximizing effective bandwidth.

7.2 Persistent Memory and Pinned Host Allocation

Memory management is overhauled to eliminate the overhead of repeated allocations during the training loop.

- **Persistent Buffers:** The `DeviceMemory` structure allocates all required buffers, including weights, hidden states, and predictions, at initialization. This eliminates per-batch `cudaMalloc` and `cudaFree` calls [5].
- **Pinned Host Memory:** The use of `cudaMallocHost` for matrix data enables the use of Page-Locked (Pinned) memory. This allows for direct memory access (DMA) transfers, significantly increasing the bandwidth between the host and device compared to standard `malloc` [6].

7.3 Stream-Based Concurrency and Kernel Fission

To further reduce idle GPU time, the implementation introduces asynchronous execution and operation fission.

- **Asynchronous Streams:** The implementation of `cudaStream_t` allows for the overlapping of data uploads with kernel execution, effectively hiding transfer latencies [3].
- **Kernel Fission:** To maximize GPU utilization, the forward pass is decomposed into multiple specialized kernels. This approach allows operations previously handled by the CPU to be offloaded to the device. By breaking the original monolithic process into discrete kernels.

7.4 Tiled Shared Memory

To mitigate global memory latency and ensure scalability for high-dimensional datasets, performance-critical kernels utilize tiled shared memory.

- **Application in Forward Pass:** In our implementation, shared memory is used in both the hidden and output layer computations (`forward_hidden_kernel` and `forward_output_kernel`) to hold input and weight tiles, allowing threads to efficiently reuse data and minimize repeated global memory accesses [1].

8 Motivations

The key motivations for this work are:

- **High PCIe Overhead:** Frequent host-device transfers slow down execution.

- **Inefficient Memory Access:** Non-coalesced global memory accesses reduce effective bandwidth.
- **Blocking Synchronizations:** Which prevents overlap of computation and data transfer.
- **Partial CPU Computation:** Operations like activation functions and memory allocations on the host limit GPU utilization.
- **Scalability Challenges:** Baseline approach underperforms for medium and large datasets.
- **Opportunity for GPU Concurrency:** Leveraging streams, shared memory, and pinned memory can maximize hardware utilization.

9 Results and Performance Evaluation

This section presents a comprehensive performance analysis. We evaluate the baseline implementation against two levels of optimization: one focusing on asynchronous execution and one combining asynchronicity with localized memory caching .

9.1 Experimental Setup

The evaluation was performed on an NVIDIA T4 GPU using google Colab platform. The dataset scales are defined as described in Section 2.3 . All configurations utilized a batch size of 256 and were trained for 100 epochs.

9.2 Execution Time Comparison

The following tables summarize the training execution times (in seconds) for each implementation as a function of thread block size.

Data Size	2 Thr	4 Thr	8 Thr	16 Thr	32 Thr
Small	0.4208	0.4161	0.3979	0.4867	0.4028
Medium	4.2234	3.2342	3.1557	3.5062	3.3030
Large	35.9853	34.0710	33.4081	31.0522	34.9211

Table 2: Baseline Implementation Results (Synchronous, Global Memory)

Data Size	2 Thr	4 Thr	8 Thr	16 Thr	32 Thr
Small	0.2250	0.1967	0.1923	0.1926	0.2190
Medium	2.1483	1.9845	2.4001	1.9884	2.6388
Large	23.0969	20.6945	19.8967	20.7024	22.6719

Table 3: Tier 1 Optimization: Pre-transposed Weights

Data Size	2 Thr	4 Thr	8 Thr	16 Thr	32 Thr
Small	0.1847	0.2168	0.1792	0.1817	0.2708
Medium	1.8271	2.4099	1.8395	2.2299	2.0439
Large	19.9671	18.5923	22.2569	18.6634	19.3986

Table 4: Tier 2 Optimization : Asynchronous streams

Data Size	2 Thr	4 Thr	8 Thr	16 Thr	32 Thr
Small	0.2236	0.1851	0.1729	0.1802	0.1741
Medium	2.5152	1.8105	1.7297	1.7128	1.8043
Large	19.9942	19.4521	18.2792	17.8190	19.8108

Table 5: Tier 3 Optimization : Asynchronous Streams and Shared memory.

The efficiency of the proposed optimizations can be clearly observed by comparing the execution times across the four tables for different data sizes and thread configurations:

- **First Optimization – Tier 1: Cache Locality and Persistent Memory (Table 3):** Largest improvement; execution times drop significantly for all datasets , with a speedup of x1,54 for the large dataset .
- **Second Optimization – Tier 2: Streams (Table 4):** Adds modest gains over the first optimization . Some performance outliers were observed in specific configurations , notably with 8-thread blocks on the large dataset .
- **Third Optimization – Tier 3: Streams and Tiled Shared Memory (Table 5):** Slight additional improvement, mainly for Large dataset , with small benefits for Medium and Small datasets.
- **Overall Trend:** Across all thread counts and data sizes, Tier 1 Optimization delivers the largest performance improvement. Tier 2 (Tier 1 + Asynchronous Streams) and Tier 3 (Tier 2 + Tiled Shared Memory), provide smaller but consistent enhancements.

The efficiency of the proposed tiled approach is best visualized in the comparison of the Medium and Large scales in Figure 3. Specifically, the reduction in global memory traffic via shared memory caching allows the Large dataset to reach a peak execution time of 17.81s, outperforming the baseline by nearly 57%.

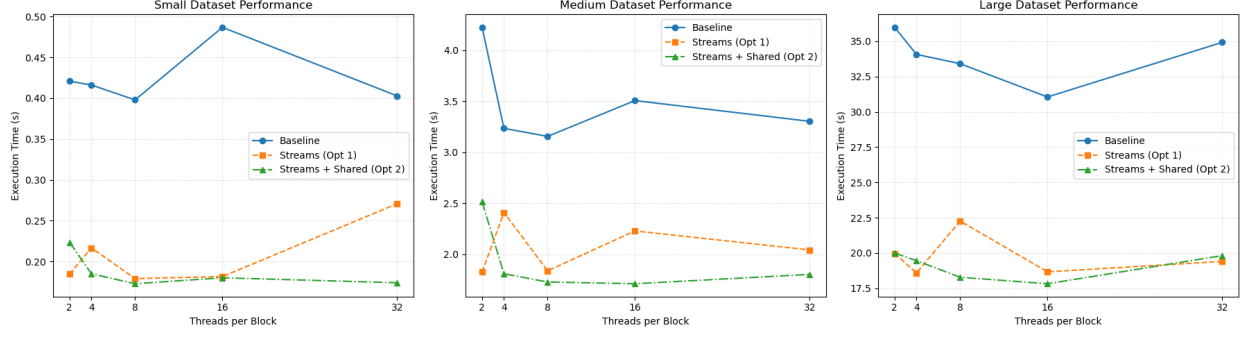


Figure 3: Comparative of Training Execution Times: Baseline vs. Streams (tier 2) vs. Streams + Shared Memory (tier 3) across different dataset scales.

9.3 Discussion of Findings

The transition from a synchronous baseline to a stream-optimized tiled implementation reveals several key performance insights:

- Impact of Tier 1 Optimization (Cache Locality + Persistent Memory):** Comparing Table 2 and Table 3, we see that pre-transposing weights for cache locality and using persistent buffers with pinned host memory deliver the largest single-step performance improvement. For the Large dataset, execution time drops from 35.98s to 19.90–23.09s, reducing latency by approximately 34–36% and achieving an acceleration of about 1.56–1.75 $\times$. This optimization improves memory access patterns, reduces global memory overhead, and significantly lowers latency, particularly for the Large dataset.
- Impact of Concurrency (Streams):** Comparing Table 2 and Table 4, we observe a massive reduction in latency (approx. 40–45% corresponding to an acceleration of approximately 1.55–1.90 $\times$ for the Large dataset). This confirms that eliminating per-batch `cudaMalloc` calls and using pinned memory to overlap transfers with computation resolves the primary PCIe bottleneck.
- Efficiency of Shared Memory Tiling:** The addition of Shared Memory (Table 5) provides the absolute best performance for the Large dataset at 16 threads (17.81s). This represents a cumulative speedup of 1.74x over the best baseline result (31.05s). The tiling effectively mitigates global memory latency by caching weight matrices in the L1-speed shared buffer.
- Thread Granularity Sweet Spot:** For the optimized shared memory version, the 16-thread configuration consistently performed best for larger workloads. At 32 threads, we observe a performance regression (19.81s), likely due to high register pressure and shared memory bank conflicts which decrease overall multiprocessor occupancy.

- **Performance Degradation at Low Thread Configurations:** configurations with 2 and 4 threads exhibit higher execution times (19.99 s and 19.45 s, respectively) compared to the 16-thread configuration. At such low thread counts, the GPU is underutilized, as there are insufficient active warps to effectively hide global memory latency and instruction latency.
- **Small Scale Stability:** For the Small dataset, all optimized versions converge around 0.17s–0.18s, suggesting that at this scale, the execution time is limited by the fixed overhead of kernel launches rather than memory throughput.

10 Conclusion

This study evaluated the performance of a CUDA-based neural network training pipeline, progressing from a naive baseline to a highly optimized implementation using asynchronous streams and tiled shared memory. Our results on the NVIDIA T4 GPU demonstrate that:

- **Memory Latency vs. Compute:** In the context of mini-batch SGD, the bottleneck is primarily dictated by host-device data transfer and synchronization rather than raw floating-point performance. By implementing pinned memory and persistent device buffers, we achieved an immediate acceleration of over 1.7x.
- **Architectural Efficiency:** The integration of shared memory tiling further optimized the memory hierarchy, allowing the system to achieve its peak efficiency of 17.81s for a large dataset at a 16-thread granularity.
- **Optimum Configuration:** We identified that while high occupancy is generally desired, increasing thread blocks beyond a certain threshold (e.g., 32 threads) results in diminished returns due to register pressure and bank conflicts.

Ultimately, the transition from a "co-processor" model to an integrated device-centric pipeline effectively minimized the PCIe bottleneck, providing a robust foundation for deep learning tasks on GPGPU architectures.

11 Future Perspectives

While the current optimizations significantly improved forward-pass efficiency, it still performs weight updates and gradient calculations on the CPU. Moving the entire backpropagation logic to the GPU would eliminate the remaining data transfers between epochs, potentially yielding another 2x–3x speedup. Further improvements could be achieved through leveraging optimized CUDA libraries such as cuBLAS and cuDNN [7], as well as mixed-precision training could further improve throughput for large datasets.

References

- [1] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.
- [2] Brouthen, K., & Akeb, A. (2025). *Exploring Parallelization of Shallow Neural Network using Cuda*. ESI Student Report.
- [3] David B. Kirk and Wen-mei W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 3rd Edition, Morgan Kaufmann, 2016.
- [4] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, 2nd Edition, Addison-Wesley, 2003.
- [5] NVIDIA Corporation. (2024). *CUDA C++ Programming Guide*. NVIDIA Doc.
- [6] Dmitrij Tichonov. (2024). *CUDA Series: Memory and Allocation*. Medium.
- [7] NVIDIA Corporation. (2024). *NVIDIA cuDNN Developer Guide*. NVIDIA Doc.